Python for Artificial Intelligence

Day 6: Functions I & II + Comprehensions

Month 1, Week 2 | Python Mastery

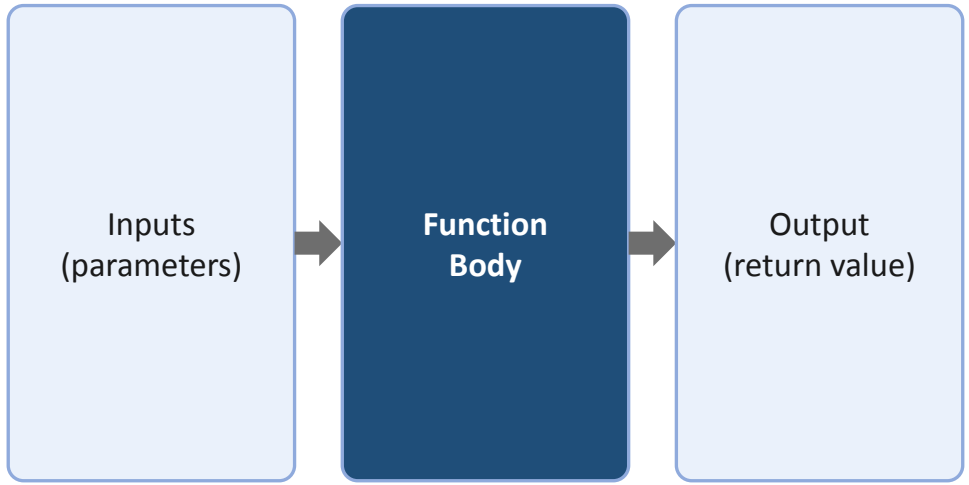
Today's Agenda

- Hour 1: Defining and calling functions, parameters, return values, default & keyword arguments
- Hour 2: *args and **kwargs, local vs. global scope
- 15-Minute Break
- Hour 3: List and dictionary comprehensions, and combining them with functions

Hour 1: Functions

Why Do We Need Functions?

- A function is a named, reusable block of code that does one job
- Instead of copy-pasting the same code, you write it once and call it whenever you need it
- Functions take inputs (parameters), do something with them, and can hand back a result (the return value)
- This is the single biggest step toward writing programs that scale beyond a few lines



Defining and Calling a Function

def starts a function definition. Calling it later runs its body.

```
def greet():
    print("Hello, welcome to the course!")

greet()      # calling the function runs its body
greet()      # call it again, as many times as you like
```

Parameters and Return Values

A parameter lets a function accept input. return sends a value back to the caller.

```
def add(a, b):
    result = a + b
    return result

total = add(5, 3)
print(total)          # 8
```

def: Full Syntax

SYNTAX

```
def function_name(parameter1, parameter2):
    # function body
    return value
```

PARAMETERS

`function_name` The name you call the function by. Follows the same naming rules as variables.

`parameter1, parameter2` Named inputs the function expects. Optional - a function can take zero parameters.

`return value` Optional. Sends a value back to wherever the function was called.

YOU CAN

✓ Call a function as many times as you like once it's defined

✓ Have a function with no parameters and no return statement

✓ Return any type: a number, a string, a list, a dictionary

✓ Use the returned value immediately: print(add(2, 3))

YOU CANNOT

✗ Call a function before it has been defined earlier in the file

✗ Use a value the function never returned - a function with no return gives back None

✗ Expect code after return to run - return exits the function immediately

Returns: Whatever follows the return keyword, or None if there is no return statement.

Default Arguments

Give a parameter a fallback value, used only if the caller doesn't supply one.

```
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

greet("Ali")                    # Hello, Ali!
greet("Sara", "Welcome")        # Welcome, Sara!
```

Keyword Arguments

Pass arguments by name instead of position - order stops mattering.

```
def describe_student(name, age, city):
    print(f"{name}, age {age}, from {city}")

describe_student(name="Ayesha", city="Karachi", age=21)
# works even though the order doesn't match the definition
```

THINK & PRACTICE

Quick Check

What does calling greet("Bilal") print, given:

```
def greet(name, greeting="Hi"):
    print(f"{greeting}, {name}!")
```

ANSWER

Quick Check - Answer

```
Hi, Bilal!

# greeting was not supplied, so it falls back
# to its default value, "Hi"
```

EXAMPLE 1: Practice - Simple Calculator Function

Type this yourself and run it.

```
def calculate(num1, num2, operation):
    if operation == "add":
        return num1 + num2
    elif operation == "subtract":
        return num1 - num2
    else:
        return None

print(calculate(10, 4, "add"))        # 14
print(calculate(10, 4, "subtract"))   # 6
```

EXAMPLE 2, INTERVIEW FAVORITE: Prime Checker

Type this yourself and run it.

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True

print(is_prime(7))    # True
print(is_prime(10))   # False
```

THINK & PRACTICE

Think & Practice

Write a function called is_even(number) that returns True if the number is even, and False otherwise. Call it with a few different numbers to test it.

ANSWER

Answer

```
def is_even(number):
    return number % 2 == 0

print(is_even(4))    # True
print(is_even(7))    # False
```

THINK & PRACTICE

Think & Practice (Harder), Interview Style

Write a function factorial(n) that returns the factorial of n (n! = n × (n-1) × ... × 1), using a loop - not recursion. factorial(0) should return 1.

ANSWER

Answer

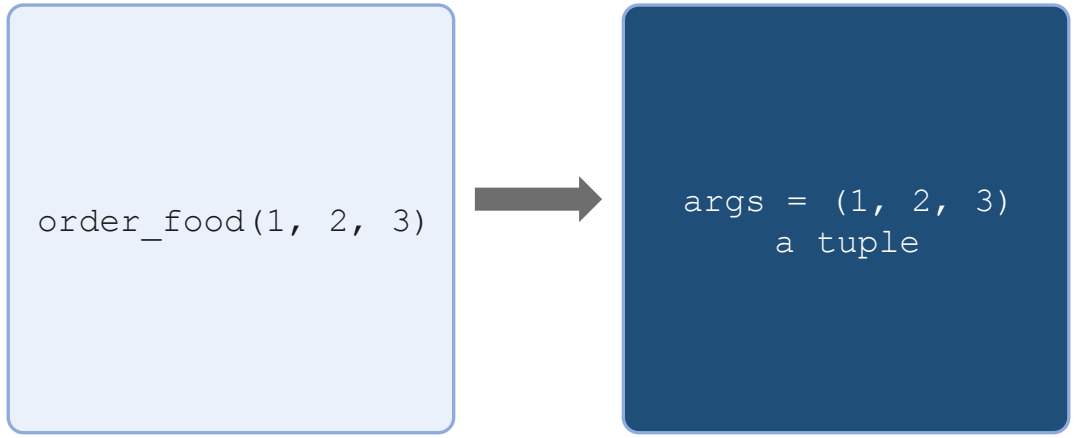
```
def factorial(n):
    result = 1
    for i in range(1, n + 1):
        result = result * i
    return result

print(factorial(5))   # 120
print(factorial(0))   # 1
```

Hour 2: Flexible Arguments & Scope

*args: Any Number of Positional Arguments

- Sometimes you don't know in advance how many arguments a function will be called with
- *args collects any number of positional arguments into a tuple inside the function
- The name args is just a convention - the * is what matters
- Useful for functions like sum-all, print-all, or anything that should accept a flexible number of inputs



Using *args

Every extra positional argument gets collected into the args tuple.

```
def total_price(*args):
    return sum(args)

print(total_price(10, 20))          # 30
print(total_price(10, 20, 5, 8))    # 43
```

*args: Full Syntax

SYNTAX

```
def function_name(*args):
    # args is a tuple of all positional arguments passed in
```

PARAMETERS

`*args` Collects any number of extra positional arguments into a tuple named args.

YOU CAN

- ✓ Call the function with 0, 1, or any number of arguments
- ✓ Loop over args like any other tuple: for a in args:
- ✓ Combine *args with normal named parameters before it

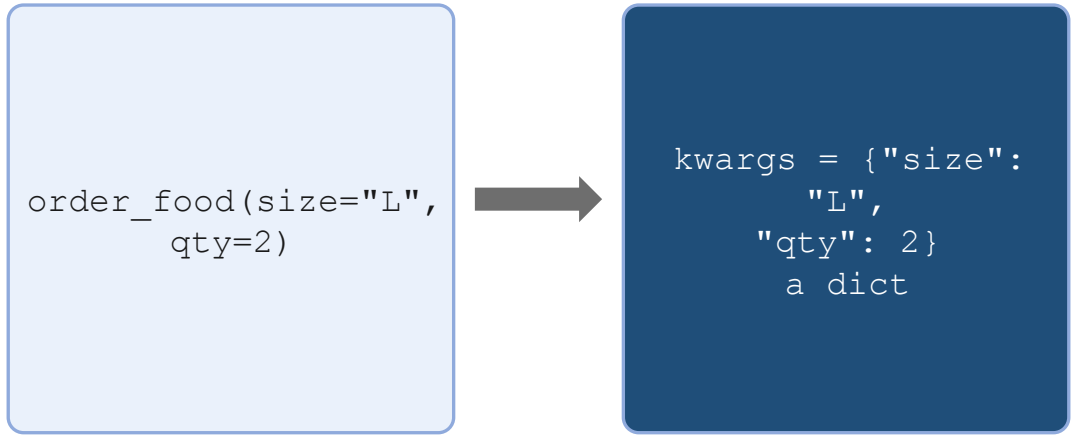
YOU CANNOT

- ✗ Access args by a keyword - it only ever collects positional arguments
- ✗ Unpack it back into named variables automatically

Returns: Nothing on its own - args is just a tuple you use inside the function body.

**kwargs: Any Number of Keyword Arguments

- **kwargs works the same way, but for keyword arguments
- It collects them into a dictionary inside the function, with argument names as keys
- Useful when a function should accept optional, named settings without listing every one upfront



Using **kwargs

Every extra keyword argument gets collected into the kwargs dictionary.

```
def describe_student(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

describe_student(name="Ali", age=22, city="Lahore")
```

**kwargs: Full Syntax

SYNTAX

```
def function_name(**kwargs):
    # kwargs is a dict of all keyword arguments passed in
```

PARAMETERS

`**kwargs` Collects any number of extra keyword arguments into a dictionary named kwargs.

YOU CAN

✓ Call the function with any set of named arguments, in any order

✓ Use kwargs.items(), kwargs.get(key) like any other dictionary

✓ Combine **kwargs with normal parameters and *args in the same function

YOU CANNOT

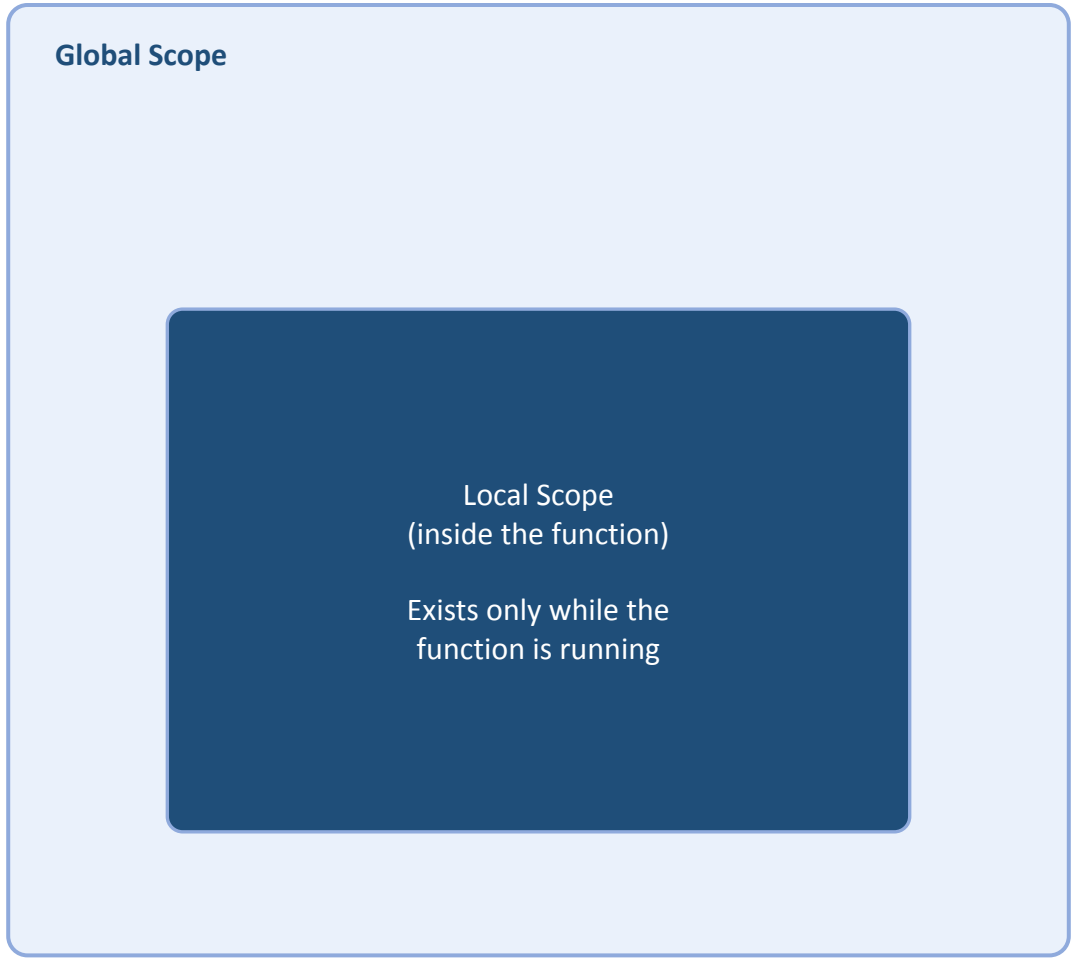
✗ Pass a positional (unnamed) argument into **kwargs

✗ Rely on a particular order - dictionaries preserve insertion order but kwargs isn't meant to be positional

Returns: Nothing on its own - kwargs is just a dictionary you use inside the function body.

Local vs. Global Scope

- A variable created inside a function is local - it only exists while that function is running
- A variable created outside any function is global - it exists for the whole program
- A function can read a global variable, but assigning to a name inside a function creates a new local one by default
- Keeping variables local (rather than everything global) is what makes functions safe to reuse



Scope in Practice

Notice how the local x inside change_x() does not affect the global x.

```
x = 10   # global

def change_x():
    x = 5   # this creates a new LOCAL x
    print("Inside function:", x)

change_x()
print("Outside function:", x)
# Inside function: 5
# Outside function: 10
```

THINK & PRACTICE

Quick Check

Given:

```
count = 100
def reset():
    count = 0
    print(count)

reset()
print(count)
```

What gets printed, in order?

ANSWER

Quick Check - Answer

```
0
100

# reset() creates its own local 'count',
# the global 'count' is never touched
```

THINK & PRACTICE

Think & Practice

Write a function add_all(*args) that returns the sum of any number of arguments passed to it. Test it with 2 numbers, then with 5 numbers.

ANSWER

Answer

```
def add_all(*args):
    return sum(args)

print(add_all(3, 7))
print(add_all(1, 2, 3, 4, 5))
```

THINK & PRACTICE

Think & Practice (Harder), Interview Style

Write a function print_profile(**kwargs) that prints each key-value pair it receives as "key: value", one per line, and also prints how many pieces of information were given.

ANSWER

Answer

```
def print_profile(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")
    print("Fields given:", len(kwargs))

print_profile(name="Hina", age=24, city="Multan")
```

15-Minute Break

Stretch, grab a coffee. We'll resume shortly.

Hour 3: List & Dictionary Comprehensions

Why Comprehensions?

- A comprehension builds a new list (or dictionary) from an existing sequence, in one line
- It's the same idea as a for-loop that appends to a list - just a more compact, Pythonic way of writing it
- Not every loop should become a comprehension - use it when the transformation is simple

The Loop Way

```
squares = []
for n in numbers:

squares.append(n *
n)
```



The Comprehension Way

```
squares = [n * n
  for n in numbers]
```

List Comprehension: Full Syntax

SYNTAX

```
[expression for item in iterable]
[expression for item in iterable if condition]
```

PARAMETERS

`expression` What to compute for each item - e.g. n * n.

`item` The loop variable, same role as in a for loop.

`iterable` Any sequence to loop over: a list, range(), a string.

`if condition` Optional. Only items where this is True are included.

YOU CAN

✓ Replace a simple for-loop-and-append pattern in one readable line

✓ Filter items with an if clause: [n for n in nums if n > 0]

✓ Nest a comprehension inside another function call: sum([n*n for n in nums])

Returns: A new list. The original iterable is never changed.

YOU CANNOT

✗ Use break, continue, or multiple statements inside it - it's an expression, not a full loop block

✗ Easily debug a very long, deeply nested comprehension - if it's hard to read, write the loop instead

EXAMPLE 1: Squares

Type this yourself and run it.

```
numbers = [1, 2, 3, 4, 5]

squares = [n * n for n in numbers]
print(squares)   # [1, 4, 9, 16, 25]
```

EXAMPLE 2: Filtering With a Comprehension

Add an if clause to keep only some items.

```
numbers = [4, 15, 8, 23, 42, 7]

evens = [n for n in numbers if n % 2 == 0]
print(evens)   # [4, 8, 42]
```

Dictionary Comprehension: Full Syntax

SYNTAX

```
{key_expr: value_expr for item in iterable}
```

PARAMETERS

`key_expr` What to use as each dictionary key.

`value_expr` What to use as each dictionary value.

`item` The loop variable, same role as in a for loop.

YOU CAN

✓ Build a dictionary from a list in one line: {n: n*n for n in numbers}

✓ Filter with an if clause, exactly like a list comprehension

YOU CANNOT

✗ Have duplicate keys - like any dictionary, a later duplicate key overwrites an earlier one

Returns: A new dictionary.

EXAMPLE 3: Dictionary Comprehension

Type this yourself and run it.

```
numbers = [1, 2, 3, 4]

squares_dict = {n: n * n for n in numbers}
print(squares_dict)   # {1: 1, 2: 4, 3: 9, 4: 16}
```

EXAMPLE 4: Combining a Function With a Comprehension

A comprehension can call a function you already wrote.

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True

numbers = range(2, 20)
primes = [n for n in numbers if is_prime(n)]
print(primes)
```

THINK & PRACTICE

Quick Check

Rewrite this loop as a single list comprehension:

```
cubes = []
for n in range(1, 6):
    cubes.append(n ** 3)
```

ANSWER

Quick Check - Answer

```
cubes = [n ** 3 for n in range(1, 6)]
```

THINK & PRACTICE

Think & Practice

Given words = ["hi", "hello", "a", "python", "ok"], use a list comprehension to build a new list containing only the words with more than 2 characters.

ANSWER

Answer

```
words = ["hi", "hello", "a", "python", "ok"]

long_words = [w for w in words if len(w) > 2]
print(long_words)   # ['hello', 'python']
```

THINK & PRACTICE

Think & Practice (Harder), Interview Style

Write a function square_if_even(n) that returns n squared if n is even, and returns n unchanged if it's odd. Then use a list comprehension with numbers = [1,2,3,4,5,6] to build a new list by applying this function to every number.

ANSWER

Answer

```
def square_if_even(n):
    if n % 2 == 0:
        return n * n
    return n

numbers = [1, 2, 3, 4, 5, 6]
result = [square_if_even(n) for n in numbers]
print(result)   # [1, 4, 3, 16, 5, 36]
```

Home Practice: Common Interview Questions

- 1. Write a function is_palindrome(word) that returns True if a word reads the same forwards and backwards.
- 2. Write a function count_vowels(word) that returns how many vowels are in a word.
- 3. Write a function that takes *args and returns the largest value, without using max().
- 4. Given a list of numbers, use a list comprehension to build a list of only the negative numbers.
- 5. Write a function that accepts **kwargs of subject: marks and returns the average mark, using values from the dictionary it builds.

Key Takeaways

- Functions let you write code once and reuse it, with parameters for input and return for output
- Default and keyword arguments make functions flexible to call
- *args and **kwargs let a function accept an unknown number of positional or keyword arguments
- A comprehension is a compact way to build a new list or dictionary from an existing one
- Coming up on Day 7: lambda functions, sorting with key=, and higher-order functions like map() and filter()